



SIMATS Online Programming Lab

JAVA PROGRAMMING



Lakshmi Narayanan E
192524249RD

CO1-AT3-Coding Challenge

View Only

← Back

QUESTIONS

Q1

Smart University Payroll
Management System
25 marks



Q2

Smart Vehicle Insurance Claim
System
25 marks



2/2 answered

Q1

Smart University Payroll Management System

25
marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)

Input

3

TeachingFaculty

101

Dr. Arun

CSE

50000

20

500

TechnicalStaff

102

Mr. Ravi

ECE

35000

15

300

VisitingFaculty

103

Dr. Kumar

IT

0

40

1200

Search Employee ID:

102

Expected Output

Employee Details

Employee ID : 101

Name : Dr. Arun

Department : CSE

Monthly Salary : 60000.0

Employee ID : 102

Name : Mr. Ravi

Department : ECE

Monthly Salary : 39500.0

Employee ID : 103

Name : Dr. Kumar

Department : IT

Monthly Salary : 48000.0

Highest Paid Employee

Employee ID : 101

Salary : 60000.0

Average Salary

49166.67

Employee Found

Employee ID : 102

Name : Mr. Ravi

Salary : 39500.0

Your Code

```

1 import java.util.Scanner;
2
3 class Employee {
4     private int employeeId;
5     private String employeeName;
6     private String department;
7     private double basicSalary;
8
9     public Employee() {}
10
11     public Employee(int employeeId, String employeeName, String department, double basicSalary) {
12         this.employeeId = employeeId;
13         this.employeeName = employeeName;
14         this.department = department;
15         this.basicSalary = basicSalary;
16     }
17
18     public int getEmployeeId() { return employeeId; }
19     public String getEmployeeName() { return employeeName; }
20     public String getDepartment() { return department; }
21     public double getBasicSalary() { return basicSalary; }
22

```

```

23     public void setEmployeeId(int employeeId) {
24     public void setEmployeeName(String employeeName) {
25     public void setDepartment(String department) {
26     public void setBasicSalary(double basicSalary) {
27
28     public double calculateSalary() {
29         return basicSalary;
30     }
31
32     public void displayEmployee() {
33         System.out.println("Employee ID : " + employeeId);
34         System.out.println("Name : " + employeeName);
35         System.out.println("Department : " + department);
36         System.out.println("Monthly Salary : " + calculateSalary());
37     }
38
39     public static void main(String[] args) {
40         Scanner sc = new Scanner(System.in);
41
42         int n = Integer.parseInt(readNonEmptyLine());
43         Employee[] employees = new Employee[n];
44
45         for (int i = 0; i < n; i++) {
46             String type = readNonEmptyLine();
47             int id = Integer.parseInt(readNonEmptyLine());
48             String name = readNonEmptyLine();
49             String dept = readNonEmptyLine();
50             double basicSalary = Double.parseDouble(readNonEmptyLine());
51
52             if (type.equals("TeachingFaculty")) {
53                 double teachingHours = Double.parseDouble(readNonEmptyLine());
54                 double incentivePerHour = Double.parseDouble(readNonEmptyLine());
55                 employees[i] = new TeachingFaculty(id, name, dept, basicSalary, teachingHours, incentivePerHour);
56
57             } else if (type.equals("TechnicalStaff")) {
58                 double overtimeHours = Double.parseDouble(readNonEmptyLine());
59                 double overtimeRate = Double.parseDouble(readNonEmptyLine());
60                 employees[i] = new TechnicalStaff(id, name, dept, basicSalary, overtimeHours, overtimeRate);
61
62             } else if (type.equals("VisitingLecturer")) {
63                 double lecturesHandled = Double.parseDouble(readNonEmptyLine());
64                 double paymentPerLecture = Double.parseDouble(readNonEmptyLine());
65                 employees[i] = new VisitingLecturer(id, name, dept, basicSalary, lecturesHandled, paymentPerLecture);
66             }
67         }

```

```

68
69     System.out.println("Employee Detail:");
70     for (Employee e : employees) {
71         System.out.println();
72         e.displayEmployee();
73     }
74     System.out.println();
75
76     Employee highestPaid = employees[0];
77     for (Employee e : employees) {
78         if (e.calculateSalary() > highestPaid.calculateSalary()) {
79             highestPaid = e;
80         }
81     }
82
83     System.out.println("Highest Paid Employee:");
84     System.out.println("Employee ID : " + highestPaid.getId());
85     System.out.println("Salary : " + highestPaid.calculateSalary());
86     System.out.println();
87
88     double total = 0;
89     for (Employee e : employees) total += e.calculateSalary();
90     double average = total / n;
91
92     System.out.println("Average Salary:");
93     System.out.printf("%.2f\n", average);
94     System.out.println();
95
96     // Search Employee ID: can be either a string or an integer
97     String line = readNonEmptyLine(sc);
98
99     int searchId;
100    if (line.contains("Search Employee ID: ")) {
101        searchId = Integer.parseInt(line.substring(15));
102    } else {
103        searchId = Integer.parseInt(line);
104    }
105
106    boolean found = false;
107    for (Employee e : employees) {
108        if (e.getId() == searchId) {
109            System.out.println("Employee Found:");
110            System.out.println("Employee ID : " + e.getId());
111            System.out.println("Name : " + e.getName());
112            System.out.println("Salary : " + e.calculateSalary());

```

```

113         found = true;
114         break;
115     }
116 }
117 if (!found) {
118     System.out.println("Employee Not Found");
119 }
120
121 sc.close();
122 }
123
124 private static String readNonEmptyLine(Scanner sc) {
125     while (sc.hasNextLine()) {
126         String s = sc.nextLine().trim();
127         if (!s.isEmpty()) return s;
128     }
129     return "";
130 }
131 }
132
133 class TeachingFaculty extends Employee {
134     private double teachingHours;
135     private double incentivePerHour;
136
137     public TeachingFaculty(int employeeId, String employeeName, double basicSalary, double teachingHours, double incentivePerHour) {
138         super(employeeId, employeeName, basicSalary);
139         this.teachingHours = teachingHours;
140         this.incentivePerHour = incentivePerHour;
141     }
142
143     @Override
144     public double calculateSalary() {
145         return getBasicSalary() + (teachingHours * incentivePerHour);
146     }
147 }
148
149
150 class TechnicalStaff extends Employee {
151     private double overtimeHours;
152     private double overtimeRate;
153
154     public TechnicalStaff(int employeeId, String employeeName, double basicSalary, double overtimeHours, double overtimeRate) {
155         super(employeeId, employeeName, basicSalary);
156         this.overtimeHours = overtimeHours;
157         this.overtimeRate = overtimeRate;

```



```

158         this.overtimeRate = overtimeRate;
159     }
160
161     @Override
162     public double calculateSalary() {
163         return getBasicSalary() + (overtimeRate *
164     }
165 }
166
167 class VisitingFaculty extends Employee {
168     private double lecturesHandled;
169     private double paymentPerLecture;
170
171     public VisitingFaculty(int employeeId, String employeeName, Department department,
172                             double basicSalary, double paymentPerLecture) {
173         super(employeeId, employeeName, department, basicSalary);
174         this.lecturesHandled = lecturesHandled;
175         this.paymentPerLecture = paymentPerLecture;
176     }
177
178     @Override
179     public double calculateSalary() {
180         return getBasicSalary() + (lecturesHandled * paymentPerLecture);
181     }
182 }

```

Input

```

3
TeachingFaculty
101
Dr. Arun

```

Output

```

Employee Details

Employee ID : 101
Name : Dr. Arun
Department : CSE
Monthly Salary : 60000.0

```

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.
